Bento

Arquitetura do banco de dados

Diagrama de relacionamentos, todas as tabelas com campos, tipos, chaves e restrições, índices, decisões de arquitetura, operação e caminhos para escalar o PostgreSQL do painel.

Versão 1.6.0
Novembro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

1.6 **O banco na versão 1.6.0** **5**

D.1 O que muda e o que não muda 5

D.2 As consultas do conjunto analítico 5

D.3 Formato enviado ao Python 6

D.4 Minimização de dados pessoais 7

D.5 Volumes, limites e desempenho 7

D.6 Como escalar 7

1.5 **O banco na versão 1.5.0** **9**

N.1 Resumo da migração 007 9

N.2 Diagrama das novidades 10

N.3 usuarios: colunas do segundo fator 11

N.4 codigos_recuperacao 11

N.5 preferencias_usuario.layout_visao_geral 11

N.6 alertas_metricas 12

N.7 disparos_alerta 13

N.8 Categorias, permissões e configuração 13

N.9 Fluxos de dados novos 14

N.10 Operação e escala 14

01 **Visão geral** **16**

1.1 PostgreSQL ou PGlite 16

1.2 Migrações 16

1.3 Convenções 18

1.4 Tabelas por área 18

1.5 Novidades da versão 1.4.0 18

02 **Diagrama de relacionamentos** **20**

2.1 Relacionamentos 20

2.2 Diagrama 21

03 **Acesso e usuários** **23**

3.1 papeis 23

3.2 permissoes 23

3.3 papel_permissoes 24

3.4 usuarios 25

3.5 usuario_permissoes 27

Copyright (©) 2026 Erik Pimenta

3.6 preferencias_usuario 27
3.7 preferencias_notificacao 28
3.8 sessoes 29
3.9 chaves_api 30
3.10 notas_usuario 31

04 **Negócio** **33**
4.1 planos 33
4.2 assinaturas 34
4.3 transacoes 35
4.4 metas_mensais 37

05 **Tráfego** **38**
5.1 trafego_diario 38
5.2 trafego_paises 39

06 **Comunicação e relatórios** **40**
6.1 notificacoes 40
6.2 notificacao_estados 41
6.3 relatorios_agendados 42
6.4 relatorios 43

07 **Sistema** **45**
7.1 integracoes 45
7.2 entregas_integracao 46
7.3 auditoria 47
7.4 configuracoes_sistema 48
7.5 migracoes 49

08 **Consultas e índices** **50**
8.1 Consultas principais 50
8.2 Índices parciais 51
8.3 Desempenho medido 51

09 **Decisões de arquitetura** **52**
9.1 Um código, dois motores 52
9.2 Sessões revogáveis 52
9.3 Segredos só como hash ou cifrados 52
9.4 Tempo real pelo próprio banco 52
9.5 Difusão barata de avisos 52
9.6 Relatórios dentro do banco 53
9.7 Histórico que sobrevive 53

Copyright (©) 2026 Erik Pimenta

9.8 Reembolso na própria transação 53
9.9 Regras no banco, lógica na aplicação 53
9.10 Fila de entregas no próprio banco 53
9.11 Backup lógico portável 53

10 **Fluxos de dados** **54**
10.1 Login e renovação 54
10.2 Transação paga 54
10.3 Reembolso 54
10.4 Bloqueio de conta 55
10.5 Relatório agendado 55
10.6 Entrega de integração 55
10.7 Modo de manutenção 55

11 **Escalabilidade** **56**
11.1 Tabelas que mais crescem 56
11.2 Métricas pré-calculadas 56
11.3 Busca textual 57
11.4 Várias instâncias e réplicas 57

12 **Operação** **58**
12.1 Backup e restauração 58
12.2 Retenção automática 58
12.3 Evoluindo o esquema 59
12.4 Consultas úteis 59

13 **Dados de demonstração** **60**
13.1 Volume 60
13.2 Contas da equipe 61
13.3 Desligando a demonstração 61

Copyright (©) 2026 Erik Pimenta

1.6

O banco na versão 1.6.0

A 1.6.0 não altera o esquema: não há migração nova. O motor analítico em Python não se conecta ao banco; quem lê os dados é a API, com três consultas somente leitura que funcionam igual no PostgreSQL e no PGlite. Este capítulo mostra essas consultas, o formato do conjunto enviado ao Python, como os dados pessoais ficam de fora e como escalar quando a base crescer.

D.1 O que muda e o que não muda

Item	Na 1.6.0
Migrações	Continuam de 001 a 007. `npm run diagnostico` deve listar 7 migrações aplicadas
Tabelas lidas	`planos`, `assinaturas`, `usuarios` (só `canal` e `ultimo_acesso`), `transacoes`, `trafego_diario`
Tabelas escritas	Nenhuma pelo motor. A API grava só a linha `inteligencia.exportada` em `auditoria` quando alguém baixa o Excel
Estado do motor	Nenhum: sem banco, sem disco, sem fila. O resultado fica 10 minutos na memória da API
Backup e retenção	Sem mudanças: não há dado novo para guardar
Importação por Excel	Usa exatamente a mesma gravação da importação por CSV (`usuarios` e `auditoria`)

D.2 As consultas do conjunto analítico

O módulo `api/src/modulos/ciencia/conjunto.ts` monta o conjunto em três etapas. As datas são convertidas para o fuso da empresa no próprio banco (`at time zone`), então o Python recebe dias locais prontos, sem fuso para interpretar.

Copyright (©) 2026 Erik Pimenta

```
select a.id::text as id, a.usuario_id::text as usuario, a.plano, a.ciclo, a.metodo,
       u.canal, a.valor, a.status,
       (a.iniciada_em   at time zone $1)::date::text as inicio,
       (a.teste_ate     at time zone $1)::date::text as teste_ate,
       (a.convertida_em at time zone $1)::date::text as convertida,
       (a.cancelada_em  at time zone $1)::date::text as cancelada,
       (u.ultimo_acesso at time zone $1)::date::text as ultimo_acesso
  from assinaturas a
  join usuarios u on u.id = a.usuario_id
 order by a.iniciada_em desc
 limit $2                              -- 100.001: o excedente marca "amostra limitada"
```

```
select assinatura_id::text as assinatura,
       (criado_em at time zone $1)::date::text as dia, status, valor, tipo
  from transacoes
 where assinatura_id is not null
   and criado_em >= now() - make_interval(months => $2)   -- 25 meses
 order by criado_em desc
 limit $3                              -- 600.001
```

A terceira etapa reaproveita `carregarDiario`, a mesma agregação diária da visão geral, sobre `transacoes`, `usuarios` e `trafego_diario` dos últimos 400 dias. As três leituras usam índices que já existiam: `assinaturas_iniciada_em`, `transacoes_criado_em` e a chave primária de `trafego_diario`.

D.3 Formato enviado ao Python

Campo	Formato	Origem
`referencia`	Data `AAAA-MM-DD`	Hoje no fuso da empresa
`planos[]`	`codigo`, `nome`, `preco_mensal`, `preco_anual` (centavos)	`planos`
`diario[]`	`dia`, `receita`, `reembolsos`, `pagas`, `falhas`, `novos`, `visitantes`, `cadastros`, `conversoes`	`carregarDiario`
`assinaturas[]`	`id` (uuid da assinatura), `plano`, `ciclo`, `metodo`, `canal`, `valor`, `status` e as cinco datas	`assinaturas` + `usuarios`
`eventos[]`	Tupla `[índice da assinatura, dia, status, valor, tipo]`	`transacoes`

Os eventos usam a posição da assinatura na lista em vez do uuid para encolher o conjunto: com os dados de demonstração, ele cai para cerca de 1 MB. Do lado do Python, o Pydantic confere cada campo (tipos, valores permitidos de `status`, `ciclo` e `metodo`, limites de tamanho das listas) e recusa campos desconhecidos.

Copyright (©) 2026 Erik Pimenta

D.4 Minimização de dados pessoais

O conjunto não leva nome, e-mail, telefone, empresa, cidade nem o id do usuário: só o id da assinatura. A API guarda, só em memória, o par assinatura → usuário. Quando a resposta do motor cita clientes (os 60 de maior risco, até 8 exemplos por segmento e até 5 exemplos por verificação de qualidade), a API busca só os nomes, com uma consulta pontual, e os junta à resposta apenas para quem tem `analytics.ver` e `usuarios.ver`; os demais recebem as análises sem nomes.

```
select u.id::text as id, u.nome
  from usuarios u
 where u.id in (select jsonb_array_elements_text($1::jsonb)::uuid)
```

A lista vai como JSON e é expandida no banco, o que funciona igual no PostgreSQL e no PGlite e mantém a consulta parametrizada. Para o Excel, a API envia ao motor só os nomes dos clientes citados (e só quando quem exporta pode ver usuários), nunca os e-mails.

D.5 Volumes, limites e desempenho

Limite	Valor	Onde muda
Assinaturas enviadas	100.000 mais recentes	`LIMITE_ASSINATURAS` em `conjunto.ts`
Pagamentos enviados	600.000 dos últimos 25 meses	`LIMITE_EVENTOS` e `MESES_EVENTOS`
Histórico diário	400 dias	`DIAS_DIARIO`
Corpo aceito pelo motor	96 MB (o conjunto no limite fica perto de 56 MB)	`BENTO_ANALISE_LIMITE_MB`
Espera pelo motor	60 s	`BENTO_ANALISE_TIMEOUT`
Cache do resultado	10 minutos, um cálculo por vez, recálculo forçado no máximo a cada minuto e falha guardada por 1 minuto	`BENTO_ANALISE_CACHE`

Medido com os dados de demonstração (1.807 assinaturas, 9.370 pagamentos): 0,3 s para montar o conjunto no PGlite e cerca de 1 s no motor, dos quais 0,35 s nas previsões, 0,22 s no risco de cancelamento e 0,13 s nos segmentos. O custo cresce de forma aproximadamente linear com o número de pagamentos.

D.6 Como escalar

As sugestões abaixo não fazem parte da 1.6.0; são o caminho recomendado quando a base passar de centenas de milhares de assinaturas ou o cálculo começar a pesar no banco principal.

Copyright (©) 2026 Erik Pimenta

Quando	O que fazer
Consultas do conjunto passam de alguns segundos	Crie o índice `transacoes (assinatura_id, criado_em) include (status, valor, tipo)` para ler os pagamentos só pelo índice
Banco principal sob carga	Aponte a leitura do conjunto para uma réplica de leitura do PostgreSQL; o motor não precisa de dados ao segundo
Mais de 1 milhão de pagamentos	Mantenha uma tabela agregada por assinatura e mês (pagos, falhas, reembolsos, valor), atualizada pelo agendador, e envie o agregado em vez dos eventos
Várias instâncias da API	Troque o cache em memória por um cache compartilhado (Redis) ou grave o último resultado numa tabela `analises` com `gerado_em` e o JSON
Análises demoradas	Calcule no agendador fora do horário de pico e mostre o último resultado na página
Motor sob carga	Aumente `--workers` do Uvicorn ou as réplicas do serviço `analise`; ele não tem estado, então escala horizontalmente

```
create index concurrently if not exists transacoes_assinatura_criado_em
    on transacoes (assinatura_id, criado_em) include (status, valor, tipo)
 where assinatura_id is not null;
```

> Ao criar índices ou tabelas novas, faça por uma migração numerada (008 em diante) para que o PGlite, o backup lógico e o diagnóstico continuem sabendo de tudo.

Copyright (©) 2026 Erik Pimenta

1.5

O banco na versão 1.5.0

A migração `007_dois_fatores_alertas.sql` acrescenta o segundo fator do login, os alertas de métricas com o histórico de disparos e o layout da visão geral nas preferências. Nada existente muda de forma: são colunas novas que aceitam nulo, três tabelas novas, duas restrições ampliadas e dados de base.

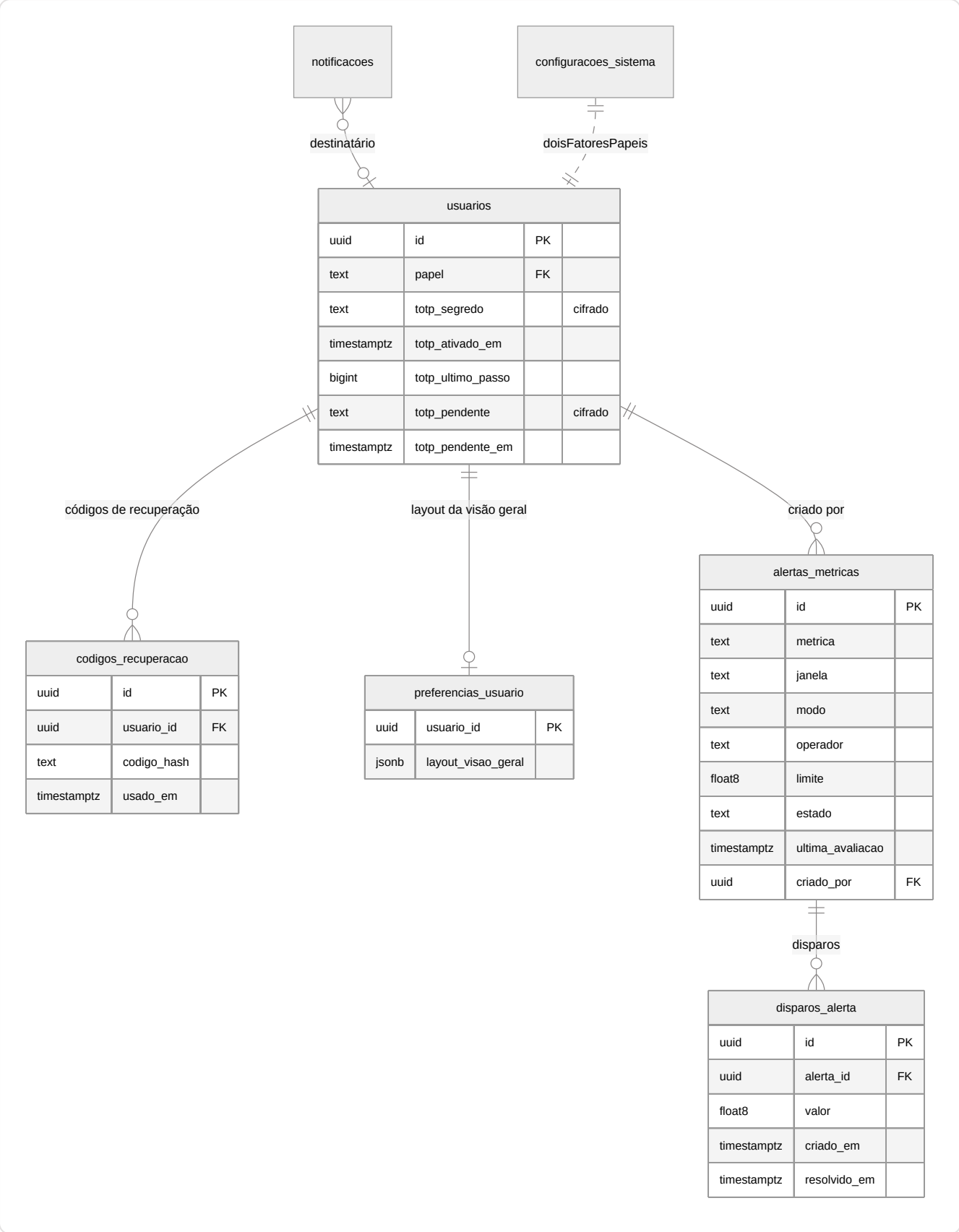
N.1 Resumo da migração 007

Tipo	Objeto	O que faz
Colunas	`usuarios.totp_*`	Segredo do aplicativo autenticador (cifrado), data de ativação, último passo aceito e o segredo pendente durante a configuração
Restrição	`usuarios_totp_coerente`	Ativação só com segredo gravado
Tabela	`codigos_recuperacao`	Dez códigos de uso único por pessoa, só como hash
Coluna	`preferencias_usuario.layout_visao_geral`	Lista JSON com a ordem, o tamanho e a visibilidade dos cartões
Restrições	`notificacoes_categoria_check` e `preferencias_notificacao_categoria_check`	Recriadas com a categoria nova `metrica`
Tabela	`alertas_metricas`	Regras de alerta, com o estado e a última medição
Tabela	`disparos_alerta`	Histórico de cada disparo, com a data de normalização
Índices	4 novos	Fila de avaliação, histórico por regra, histórico recente e códigos disponíveis
Dados	`permissoes`, `papel_permissoes`, `configuracoes_sistema`	Permissões `alertas.ver` e `alertas.gerenciar` e a chave `doisFatoresPapeis`

Como as anteriores, a 007 roda uma vez, dentro da transação do `aplicarMigracoes`, protegida pela trava consultiva que impede duas instâncias de migrar ao mesmo tempo. O backup lógico já inclui as tabelas novas, porque descobre as tabelas do esquema e a ordem das chaves estrangeiras sozinho.

Copyright (©) 2026 Erik Pimenta

N.2 Diagrama das novidades



Copyright (©) 2026 Erik Pimenta

As tabelas e colunas acima se somam ao diagrama do capítulo 02. `doisFatoresPapeis` não é chave estrangeira: é uma lista JSON em `configuracoes_sistema` com os códigos dos papéis.

N.3 usuarios: colunas do segundo fator

Coluna	Tipo	Nulo	Descrição
`totp_segredo`	text	sim	Segredo base32 do aplicativo, cifrado com AES-256-GCM (`enc:v1:…`), chave derivada do `BENTO_SEGREDO_JWT`
`totp_ativado_em`	timestamptz	sim	Quando a verificação foi ativada; nulo significa desligada
`totp_ultimo_passo`	bigint	sim	Último passo de 30 s aceito; códigos do mesmo passo ou anteriores são recusados
`totp_pendente`	text	sim	Segredo cifrado gerado em `iniciar` e ainda não confirmado
`totp_pendente_em`	timestamptz	sim	Quando o pendente foi gerado; vale por 15 minutos

A restrição `usuarios_totp_coerente` garante `totp_ativado_em is null or totp_segredo is not null`. O mapeador de usuários expõe só o booleano `doisFatores`; o segredo nunca sai do banco pela API.

N.4 codigos_recuperacao

Coluna	Tipo	Restrições	Descrição
`id`	uuid	PK, `gen_random_uuid()`	Identificador
`usuario_id`	uuid	FK `usuarios`, `on delete cascade`	Dono do código
`codigo_hash`	text	`unique (usuario_id, codigo_hash)`	HMAC-SHA256 do id da pessoa e do código normalizado, com chave derivada do `BENTO_SEGREDO_JWT`
`usado_em`	timestamptz	nulo	Quando foi usado; preenchido uma única vez
`criado_em`	timestamptz	`default now()`	Quando o lote foi gerado

Índice parcial `codigos_recuperacao_disponiveis (usuario_id) where usado_em is null`. Gerar novos códigos apaga o lote anterior e grava dez novos na mesma transação. O uso é um `update … set usado_em = now() where … and usado_em is null returning id`, atômico mesmo com dois pedidos simultâneos.

N.5 preferencias_usuario.layout_visao_geral

Coluna `jsonb` que aceita nulo, com `check (layout_visao_geral is null or (jsonb_typeof(layout_visao_geral) = 'array' and jsonb_array_length(layout_visao_geral) <= 40))`. Cada item é `{"id", "largura", "altura",`

Copyright (©) 2026 Erik Pimenta

`"oculto"}` ; a API valida o formato, os valores permitidos e a unicidade dos identificadores antes de gravar. Nulo significa o layout padrão.

```
[{"id": "receita", "largura": "dois-tercos", "altura": "alto", "oculto": false},
 {"id": "canais", "largura": "metade", "altura": "normal", "oculto": true}]
```

N.6 alertas_metricas

Coluna	Tipo	Restrições	Descrição
`id`	uuid	PK	Identificador
`nome`	text	2 a 80 caracteres	Nome da regra
`metrica`	text	`check` com as 9 métricas	`receita` , `transacoes_pagas` , `taxa_falha` , `reembolsos` , `novos_usuarios` , `cancelamentos` , `mrr` , `ticket_medio` , `conversao`
`janela`	text	`1h` , `24h` , `7d` , `30d`	Tamanho da janela, contada a partir da medição
`modo`	text	`valor` ou `variacao`	Compara o valor ou a variação contra a janela anterior
`operador`	text	`maior` ou `menor`	Acima ou abaixo; na variação, sobe ou cai
`limite`	double precision	entre -10^{12} e 10^{12}	Centavos para moeda; pontos para percentual e variação
`severidade`	text	`info` , `alerta` , `erro`	Vira o tipo da notificação
`notificar_integracoes`	boolean	`default false`	Envia `alerta.disparado` às integrações ativas
`silencio_minutos`	integer	5 a 10.080, padrão 60	Tempo mínimo entre dois disparos
`ativo`	boolean	`default true`	Pausada quando falso
`estado`	text	`normal` ou `disparado`	Estado atual da regra
`ultimo_valor`	double precision	nulo	Valor da última medição
`ultima_avaliacao`	timestamptz	nulo	Também serve de reserva na fila
`ultimo_disparo`	timestamptz	nulo	Base do silêncio
`criado_por`	uuid	FK `usuarios` , `on delete set null`	Quem criou

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Restrições	Descrição
`criado_em` / `atualizado_em`	timestamptz	`default now()`	Datas de controle

Índice parcial `alertas_metricas_fila (ultima_avaliacao nulls first) where ativo`, usado pela reserva do agendador. A aplicação limita a 50 regras por instalação.

N.7 disparos_alerta

Coluna	Tipo	Restrições	Descrição
`id`	uuid	PK	Identificador
`alerta_id`	uuid	FK `alertas_metricas`, `on delete cascade`	Regra que disparou
`valor`	double precision	obrigatório	Valor medido no disparo
`limite`	double precision	obrigatório	Limite da regra naquele momento
`mensagem`	text	1 a 300 caracteres	Texto enviado à equipe
`criado_em`	timestamptz	`default now()`	Quando disparou
`resolvido_em`	timestamptz	nulo, `>= criado_em`	Quando normalizou, foi pausada ou teve a regra alterada

Índices `disparos_alerta_lista (alerta_id, criado_em desc)` e `disparos_alerta_recentes (criado_em desc)`. Excluir a regra apaga o histórico dela; as notificações já enviadas continuam.

N.8 Categorias, permissões e configuração

- **Categoria `metrica`**: as restrições de `notificacoes.categoria` e `preferencias_notificacao.categoria` são apagadas e recriadas com a lista ampliada. Como a lista só cresce, nenhuma linha existente é afetada.
- **Permissões**: `alertas.ver` (ordem 63) e `alertas.gerenciar` (ordem 64), módulo Alertas, ligadas a `admin`, `gerente` e `analista` com `on conflict do nothing`, só para papéis que existem.
- **`configuracoes_sistema.doisFatoresPapeis`**: lista JSON de códigos de papel, inicialmente vazia. A autenticação lê essa chave na mesma consulta que valida a sessão, com o operador `?` do `jsonb`, sem uma ida extra ao banco.

Copyright (©) 2026 Erik Pimenta

N.9 Fluxos de dados novos

Login em duas etapas

1. `POST /auth/entrar` confere a senha como antes. Se `totp_ativado_em` estiver preenchido, nenhuma sessão é criada: a API assina um desafio de 5 minutos e o entrega num cookie `httpOnly`.
2. `POST /auth/dois-fatores` valida o desafio e o código. Para o aplicativo, decifra o segredo, confere ±1 passo e grava o passo com `update … where totp_ultimo_passo is null or totp_ultimo_passo < $2`. Para um código de recuperação, marca `usado_em`.
3. Uma falha soma em `tentativas_falhas` com `tentativas_falhas + 1 … returning`, atômico mesmo com pedidos em paralelo.
4. Com sucesso, zera `tentativas_falhas`, insere a sessão e registra `auth.entrou` com o método. Com erro, soma em `tentativas_falhas` e, na quinta, grava `bloqueio_temporario_ate`.

Avaliação de alertas

1. A cada minuto, o agendador reserva até 50 regras com `update alertas_metricas set ultima_avaliacao = now() where id in (select … where ativo and (ultima_avaliacao is null or ultima_avaliacao < now() - 5 min) order by ultima_avaliacao nulls first limit 50 for update skip locked) returning …`.
2. Cada par métrica e janela é medido uma vez por ciclo, direto em `transacoes`, `usuarios`, `assinaturas` e `trafego_diario`, aproveitando os índices por data.
3. Ao disparar, a mesma transação muda `estado` com `where estado = 'normal' and ativo and atualizado_em = …` e insere em `disparos_alerta`; depois vêm a notificação (com `NOTIFY` para o tempo real) e, se marcado, a fila de entregas.
4. Ao normalizar, `estado` volta a `normal` só se ainda estava `disparado`, e o disparo aberto recebe `resolvido_em`.

Previsão e insights

`GET /insights` não grava nada. Ele reaproveita `carregarDiario` (agregações por dia em transações, usuários e tráfego) para cerca de 17 semanas, as assinaturas para o MRR semanal e duas consultas curtas para canais e planos. O cálculo acontece na aplicação, sobre algumas centenas de pontos.

N.10 Operação e escala

Assunto	O que saber
Retenção	Disparos com `resolvido_em` preenchido saem depois de `BENTO_RETENCAO_ALERTAS` dias (180 por padrão), na limpeza diária e na prévia da aba Manutenção. Disparos abertos nunca são apagados pela retenção.
Crescimento	`disparos_alerta` cresce no máximo um registro por regra a cada silêncio; com 50 regras e silêncio de 1 hora, são menos de 1.200 por dia no pior caso.
Várias instâncias	A reserva por `update … returning` divide as regras entre as instâncias; a troca de estado condicionada impede disparos em dobro.

Copyright (©) 2026 Erik Pimenta

Assunto	O que saber
Custo das medições	Cada medição é uma agregação por intervalo de datas coberta pelos índices de `criado_em`. Para bases muito grandes, uma tabela de métricas diárias pré-calculadas (capítulo 11) atende alertas de 7 e 30 dias.
Troca do segredo	Trocar o `BENTO_SEGREDO_JWT` torna ilegíveis os segredos TOTP. Para recuperar, limpe as colunas `totp_*` e os códigos de quem precisa reconfigurar, ou use a redefinição pela interface.
Consulta útil	`select papel, count(*) filter (where totp_ativado_em is null) as sem_2fa from usuarios where status = 'ativo' group by papel;`

Copyright (©) 2026 Erik Pimenta

01

Visão geral

O Bento guarda tudo em um banco PostgreSQL com 24 tabelas de domínio e a tabela de controle das migrações, somando 210 campos, 26 chaves estrangeiras, 80 regras de verificação e 63 índices.

1.1 PostgreSQL ou PGlite

O mesmo SQL roda em dois motores, escolhidos na inicialização pela variável `DATABASE_URL`:

	PostgreSQL 18	PGlite 0.5
Quando	`DATABASE_URL` definida (o Docker Compose define)	`DATABASE_URL` vazia
Onde ficam os dados	No servidor PostgreSQL (volume `bento_dados-postgres` no Docker)	Na pasta `dados/pglite` (ou em memória com `BENTO_DADOS=memoria`)
Conexões	`pg.Pool` com `BENTO_POOL` conexões (padrão 10)	Uma conexão dentro do processo
Tempo real entre instâncias	`LISTEN` / `NOTIFY` no canal `bento_eventos`	Emissor em memória (instância única)
Backup	`npm run backup` com o servidor no ar, ou `pg_dump`	`npm run backup` com o servidor parado, ou o botão da aba Manutenção
Indicado para	Produção e várias instâncias	Desenvolvimento, testes e instalações pequenas

O PGlite é o próprio PostgreSQL compilado para WebAssembly, então tipos, funções, restrições e índices se comportam igual. A camada `api/src/banco/conexao.ts` expõe uma interface única (`consultar`, `executar`, `transacao`, `ouvir`, `notificar` e `fechar`) e normaliza os tipos: `int8` e `numeric` chegam como número e `date` como texto AAAA-MM-DD. A aba Status do sistema das Configurações e o `npm run diagnostico` mostram qual dos dois está em uso e a latência de uma consulta.

1.2 Migrações

Arquivo	Conteúdo
`001_esquema.sql`	Criação das 22 tabelas originais, com chaves, padrões e regras de verificação
`002_indices.sql`	Índices de consulta, incluindo os únicos e os parciais

Copyright (©) 2026 Erik Pimenta

Arquivo	Conteúdo
`003_dados_base.sql`	Papéis, permissões, permissões de cada papel, planos, integrações e configurações iniciais
`004_reembolsos_auditoria.sql`	Colunas de reembolso em `transacoes`, permissão `transacoes.reembolsar` e o índice `auditoria_acao_criado_em`
`005_notas_usuario.sql`	Tabela `notas_usuario`, índice da listagem e permissão `usuarios.anotar`
`006_operacao_preferencias.sql`	Cinco colunas novas em `preferencias_usuario`, tabela `entregas_integracao` com dois índices, índice `sessoes_expira_em`, permissão `usuarios.importar` e a chave `manutencao` em `configuracoes_sistema`

A cada inicialização, a API cria a tabela `migracoes` se ela não existir, lista os arquivos de `api/migracoes/` em ordem alfabética e aplica os que faltam, todos em uma única transação protegida por `pg_advisory_xact_lock`. Assim, várias instâncias subindo ao mesmo tempo não aplicam a mesma migração duas vezes, e uma migração com erro não deixa o banco pela metade. A sonda `/api/v1/saude/pronto` responde 503 enquanto houver migração pendente, e o diagnóstico avisa sobre migrações aplicadas que esta versão não conhece. Para mudar o esquema, crie um arquivo novo com o próximo número, como `007_estoque.sql`. Nunca altere uma migração já aplicada: a API não a roda de novo.

O que mudou na 1.4.0

```
alter table preferencias_usuario
  add column if not exists pagina_inicial text not null default 'visao-geral'
    check (pagina_inicial in ('visao-geral', 'analytics', 'usuarios', 'transacoes', 'notificacoes',
'relatorios', 'auditoria')),
  add column if not exists itens_por_pagina smallint not null default 25 check (itens_por_pagina in
(10, 25, 50, 100)),
  add column if not exists movimento text not null default 'sistema' check (movimento in ('sistema',
'reduzido')),
  add column if not exists atalhos_teclado boolean not null default true,
  add column if not exists primeiros_passos_ocultos boolean not null default false;

create table if not exists entregas_integracao (
  id uuid primary key default gen_random_uuid(),
  integracao_id uuid not null references integracoes (id) on delete cascade,
  evento text not null check (evento ~ '^[a-z_]+\.[a-z_]+$'),
  resumo text not null check (char_length(resumo) between 1 and 300),
  carga jsonb not null default '{}'::jsonb,
  status text not null default 'pendente' check (status in ('pendente', 'entregue', 'falhou')),
  tentativas smallint not null default 0 check (tentativas >= 0),
  proxima_tentativa timestamptz,
  codigo_http smallint,
  duracao_ms integer check (duracao_ms is null or duracao_ms >= 0),
  erro text check (erro is null or char_length(erro) <= 300),
  teste boolean not null default false,
  criado_em timestamptz not null default now(),
  atualizado_em timestamptz not null default now(),
  check ((status = 'pendente') = (proxima_tentativa is not null))
);
```

A migração inteira usa `if not exists` e `on conflict do nothing`, então é segura em um banco com papéis personalizados ou que já tenha parte das mudanças.

Copyright (©) 2026 Erik Pimenta

1.3 Convenções

- **Chaves primárias em uuid** com `gen_random_uuid()`, nativo desde o PostgreSQL 13. As exceções são os catálogos (papéis, permissões, planos e configurações), que usam um código em texto, e a auditoria, que usa `bigint` sequencial por causa do volume.
- **Dinheiro em centavos**, sempre `integer`. Somas grandes são feitas em ponto flutuante de 8 bytes para não estourar o inteiro de 4 bytes.
- **Datas e horas em timestamptz**, gravadas em UTC. As agregações por dia e os filtros de período convertem para o fuso de `BENTO_FUSO`. O tipo `date` só aparece onde o dado é um dia de calendário: tráfego, metas e período dos relatórios.
- **Domínios fechados com regras check** (`campo in (...)`) em vez de tipos enum. São fáceis de estender em uma migração e funcionam igual no PGlite.
- **Texto livre com limite de tamanho** por regra de verificação, como o motivo do reembolso (3 a 300 caracteres), o texto das notas (2 a 2.000) e o erro das entregas (até 300).
- **Coerência entre status e datas no banco**: uma transação paga sempre tem `pago_em`, uma conta bloqueada sempre tem `bloqueado_em` e uma entrega pendente sempre tem `proxima_tentativa`.
- **Sem gatilhos nem funções no banco**: `atualizado_em` e as demais regras de negócio ficam na aplicação, onde são testadas.
- **Nomes em português**, sem acentos, no singular para campos e no plural para tabelas.

1.4 Tabelas por área

Área	Tabelas
Acesso e usuários	`papeis` `permissoes` `papel_permissoes` `usuarios` `usuario_permissoes` `preferencias_usuario` `preferencias_notificacao` `sessoes` `chaves_api` `notas_usuario`
Negócio	`planos` `assinaturas` `transacoes` `metas_mensais`
Tráfego	`trafego_diario` `trafego_paises`
Comunicação e relatórios	`notificacoes` `notificacao_estados` `relatorios_agendados` `relatorios`
Sistema	`integracoes` `entregas_integracao` `auditoria` `configuracoes_sistema` `migracoes`

1.5 Novidades da versão 1.4.0

O que	Detalhe
Preferências de uso	`preferencias_usuario` ganhou `pagina_inicial`, `itens_por_pagina`, `movimento`, `atalhos_teclado` e `primeiros_passos_ocultos` (seção 3.6)
Entregas das integrações	Tabela nova `entregas_integracao`, histórico e fila de novas tentativas (seção 7.2)
Retenção	Índice `sessoes_expira_em` para a limpeza das sessões; a retenção automática apaga sessões, notificações, relatórios e entregas antigas (seção 12.2)
Importação	Permissão `usuarios.importar` para administradores e gerentes

Copyright (©) 2026 Erik Pimenta

O que	Detalhe
Manutenção	Chave `manutencao` em `configuracoes_sistema`, com o estado do modo somente leitura (seção 7.4)
Backup lógico	`npm run backup` e `npm run restaurar` gravam e leem todas as tabelas em NDJSON compactado, em PGlite ou PostgreSQL (seção 12.1)

Copyright (©) 2026 Erik Pimenta

02

Diagrama de relacionamentos

A tabela usuarios está no centro do modelo: quase tudo aponta para ela. No diagrama da seção 2.2, o acesso fica à esquerda, o negócio e a comunicação à direita, e as tabelas independentes em cima.

2.1 Relacionamentos

Todas as 26 chaves estrangeiras do esquema, as que apontam para `usuarios` primeiro. **Ao excluir** diz o que acontece com a linha filha quando a linha referenciada é apagada.

Campo	Referência	Card.	Obrigatório	Ao excluir	Ao atualizar
`assinaturas.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`auditoria.usuario_id`	`usuarios.id`	N:1	não	vira nulo	impede
`chaves_api.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`notas_usuario.autor_id`	`usuarios.id`	N:1	não	vira nulo	impede
`notas_usuario.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`notificacao_estados.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`notificacoes.criado_por`	`usuarios.id`	N:1	não	vira nulo	impede
`notificacoes.destinatario_id`	`usuarios.id`	N:1	não	cascata	impede
`preferencias_notificacao.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`preferencias_usuario.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`relatorios.criado_por`	`usuarios.id`	N:1	não	vira nulo	impede
`relatorios_agendados.criado_por`	`usuarios.id`	N:1	sim	cascata	impede
`sessoes.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede
`transacoes.reembolsado_por`	`usuarios.id`	N:1	não	vira nulo	impede
`transacoes.usuario_id`	`usuarios.id`	N:1	não	vira nulo	impede
`usuario_permissoes.usuario_id`	`usuarios.id`	N:1	sim	cascata	impede

Copyright (©) 2026 Erik Pimenta

Campo	Referência	Card.	Obrigatório	Ao excluir	Ao atualizar
`transacoes.assinatura_id`	`assinaturas.id`	N : 1	não	vira nulo	impede
`entregas_integracao.integracao_id`	`integracoes.id`	N : 1	sim	cascata	impede
`notificacao_estados.notificacao_id`	`notificacoes.id`	N : 1	sim	cascata	impede
`papel_permissoes.papel`	`papeis.codigo`	N : 1	sim	cascata	cascata
`usuarios.papel`	`papeis.codigo`	N : 1	sim	impede	cascata
`papel_permissoes.permissao`	`permissoes.codigo`	N : 1	sim	cascata	cascata
`usuario_permissoes.permissao`	`permissoes.codigo`	N : 1	sim	cascata	cascata
`assinaturas.plano`	`planos.codigo`	N : 1	sim	impede	cascata
`transacoes.plano`	`planos.codigo`	N : 1	não	impede	cascata
`relatorios.agendamento_id`	`relatorios_agendados.id`	N : 1	não	vira nulo	impede

2.2 Diagrama

Cada caixa mostra a chave primária (PK), as chaves estrangeiras (FK) e o total de campos da tabela. As linhas ligam a tabela referenciada à que aponta para ela, com o pé de galinha no lado “muitos”.

Copyright (©) 2026 Erik Pimenta

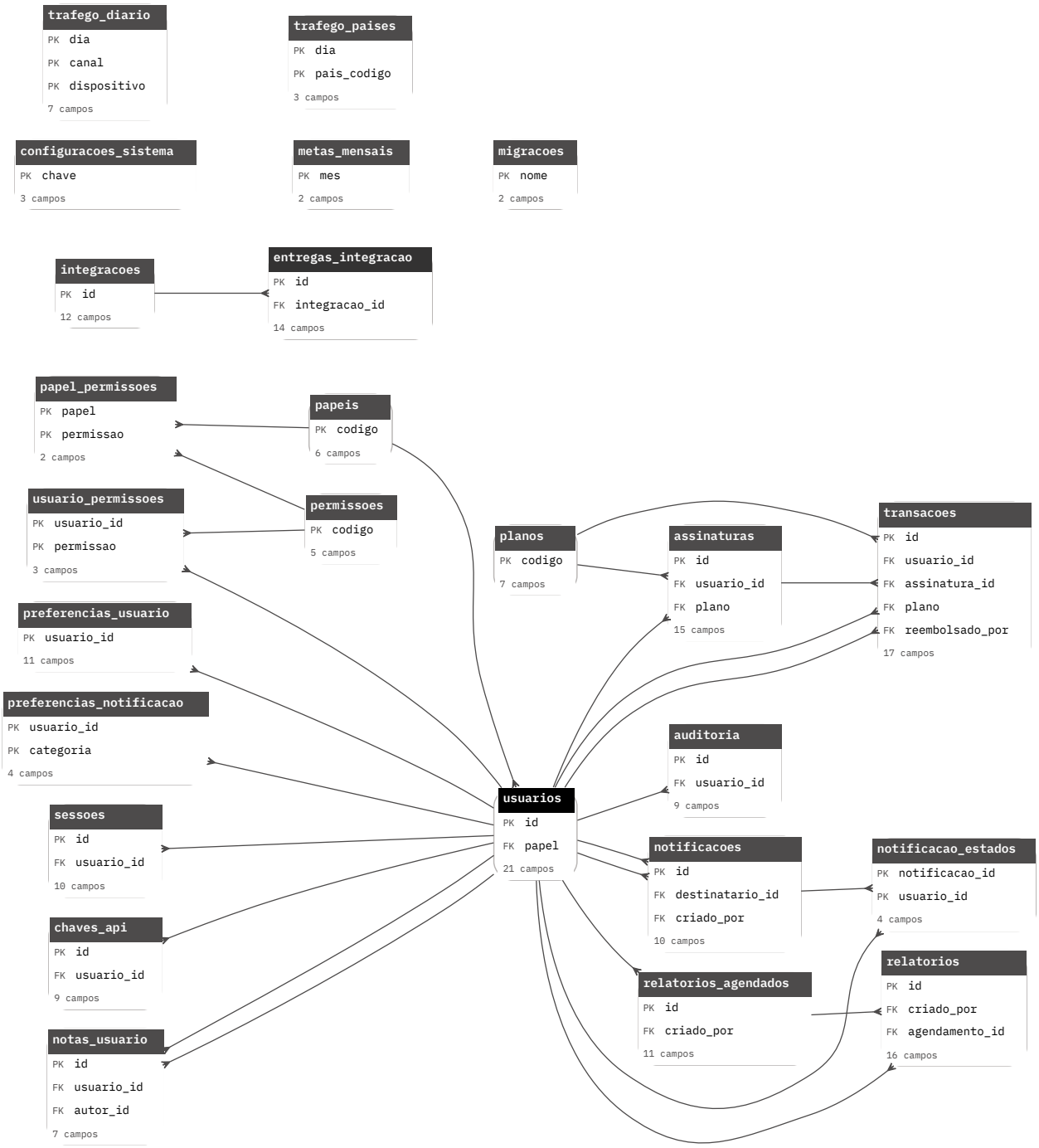


Figura 2.1 Diagrama de relacionamentos das 25 tabelas. A caixa preta é `usuarios`.

Copyright (©) 2026 Erik Pimenta

03

Acesso e usuários

Contas, papéis, permissões, sessões, chaves de API, preferências e notas internas: tudo que decide quem entra e o que cada pessoa pode fazer.

3.1 papeis

Catálogo dos papéis de acesso. Cada usuário tem exatamente um papel, que define as permissões de partida e se a conta entra no painel.

6 campos. 1 índice e 5 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`codigo` PK	text	não		Identificador do papel: admin, gerente, analista, suporte ou cliente
`nome`	text	não		Nome exibido na interface
`descricao`	text	não	''	Explicação mostrada na tela de permissões
`nivel`	smallint	não	0	Ordem hierárquica; quanto maior, mais acima na lista
`acesso_painel`	boolean	não	true	Falso impede o login no painel (clientes)
`sistema`	boolean	não	true	Marca os papéis criados pela migração base

Restrições

Tipo	Regra
verificação	`codigo ~ '^[a-z][a-z0-9_]{1,31}$'`

Índices

Índice	Colunas	Tipo	Para que serve
`papeis_pkey`	`(codigo)`	único	Chave primária

3.2 permissoes

Catálogo das 19 permissões, uma para cada ação protegida da API e da interface.

Copyright (©) 2026 Erik Pimenta

5 campos. 1 índice e 19 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`codigo` PK	text	não		Código no formato modulo.acao, usado em exigir() e pode()
`modulo`	text	não		Grupo exibido na tela de permissões
`nome`	text	não		Nome curto da permissão
`descricao`	text	não	''	O que a permissão libera
`ordem`	smallint	não	0	Posição na listagem

Restrições

Tipo	Regra
verificação	`codigo ~ '^[a-z]+\.[a-z]+$'`

Índices

Índice	Colunas	Tipo	Para que serve
`permissoes_pkey`	`(codigo)`	único	Chave primária

3.3 papel_permissoes

Permissões de cada papel. Excluir um papel ou uma permissão apaga as ligações; renomear o código propaga a mudança.

2 campos. 1 índice e 48 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`papel` PK FK	text	não		Papel que recebe a permissão → papeis.codigo
`permissao` PK FK	text	não		Permissão concedida ao papel → permissoes.codigo

Restrições

Tipo	Regra
chave estrangeira	`papel` → `papeis.codigo`, ao excluir: cascata, ao atualizar: cascata
chave estrangeira	`permissao` → `permissoes.codigo`, ao excluir: cascata, ao atualizar: cascata

Copyright (©) 2026 Erik Pimenta

Índices

Índice	Colunas	Tipo	Para que serve
`papel_permissoes_pkey`	`(papel, permissao)`	único	Chave primária

3.4 usuarios

Todas as contas: equipe e clientes da plataforma. É a tabela central do modelo, referenciada por sessões, assinaturas, transações, reembolsos, notificações, relatórios e auditoria.

21 campos. 7 índices e 2.393 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador gerado pelo banco
`nome`	text	não		Nome completo, de 2 a 120 caracteres
`email`	text	não		E-mail de login, único sem diferenciar maiúsculas e minúsculas
`senha_hash`	text	sim		Hash bcrypt da senha; nulo para clientes sem acesso ao painel
`papel` FK	text	não	'cliente'	Papel de acesso → papeis.codigo
`status`	text	não	'ativo'	ativo, bloqueado, pendente ou inativo
`cargo`	text	sim		Cargo na empresa (até 80 caracteres)
`empresa`	text	sim		Empresa da pessoa (até 120 caracteres)
`telefone`	text	sim		Telefone de contato (até 30 caracteres)
`pais`	text	sim		País (até 60 caracteres)
`cidade`	text	sim		Cidade (até 80 caracteres)
`canal`	text	sim		Canal de aquisição: Orgânico, Direto, Pago, Social, Indicação ou E-mail
`fuso`	text	não	'America/Sao_Paulo'	Fuso horário preferido
`idioma`	text	não	'pt-BR'	Idioma preferido
`tentativas_falhas`	smallint	não	0	Senhas erradas seguidas desde o último login
`bloqueio_temporario_ate`	timestamptz	sim		Fim da suspensão de 15 minutos após 5 tentativas
`bloqueado_em`	timestamptz	sim		Momento do bloqueio administrativo
`motivo_bloqueio`	text	sim		Motivo informado ao bloquear (até 300 caracteres)

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`ultimo_acesso`	timestamptz	sim		Último login bem-sucedido
`criado_em`	timestamptz	não	now()	Data do cadastro
`atualizado_em`	timestamptz	não	now()	Última alteração, definida pela aplicação

Restrições

Tipo	Regra
verificação	`(canal IS NULL) OR (canal in ('Orgânico', 'Direto', 'Pago', 'Social', 'Indicação', 'E-mail'))`
verificação	`(cargo IS NULL) OR (char_length(cargo) <= 80)`
verificação	`(status = 'bloqueado') = (bloqueado_em IS NOT NULL)`
verificação	`(cidade IS NULL) OR (char_length(cidade) <= 80)`
verificação	`(char_length(email) <= 254) AND (email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$')`
verificação	`(empresa IS NULL) OR (char_length(empresa) <= 120)`
verificação	`(motivo_bloqueio IS NULL) OR (char_length(motivo_bloqueio) <= 300)`
verificação	`(char_length(btrim(nome)) >= 2) AND (char_length(btrim(nome)) <= 120)`
verificação	`(pais IS NULL) OR (char_length(pais) <= 60)`
verificação	`status in ('ativo', 'bloqueado', 'pendente', 'inativo')`
verificação	`(telefone IS NULL) OR (char_length(telefone) <= 30)`
verificação	`tentativas_falhas >= 0`
chave estrangeira	`papel` → `papeis.codigo`, ao excluir: impede, ao atualizar: cascata

Índices

Índice	Colunas	Tipo	Para que serve
`usuarios_pkey`	`(id)`	único	Chave primária
`usuarios_criado_em`	`(criado_em)`	comum	Cadastros por período e ordenação por data
`usuarios_email_unico`	`(lower(email))`	único	Login e checagem de e-mail duplicado sem diferenciar maiúsculas
`usuarios_nome`	`(lower(nome))`	comum	Ordenação por nome
`usuarios_papel`	`(papel)`	comum	Filtro por papel

Copyright (©) 2026 Erik Pimenta

Índice	Colunas	Tipo	Para que serve
`usuarios_status`	`(status)`	comum	Filtro e contagem por status
`usuarios_ultimo_acesso`	`(ultimo_acesso DESC NULLS LAST)`	comum	Ordenação por último acesso e usuários ativos

3.5 usuario_permissoes

Exceções individuais às permissões do papel. As permissões efetivas de uma pessoa são as do papel, mais as concedidas e menos as negadas.

3 campos. 1 índice e 0 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`usuario_id` PK FK	uuid	não		Pessoa que recebe a exceção → usuarios.id
`permissao` PK FK	text	não		Permissão ajustada → permissoes.codigo
`concedida`	boolean	não		Verdadeiro concede além do papel; falso nega mesmo que o papel tenha

Restrições

Tipo	Regra
chave estrangeira	`permissao` → `permissoes.codigo`, ao excluir: cascata, ao atualizar: cascata
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`usuario_permissoes_pkey`	`(usuario_id, permissao)`	único	

3.6 preferencias_usuario

Aparência escolhida por cada pessoa, sincronizada entre computadores. Relação um para um com usuarios; sem linha, valem os padrões. Na 1.4.0 ganhou a página inicial, as linhas por página, o movimento, os atalhos de uma tecla e o guia de primeiros passos.

11 campos. 1 índice e 4 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`usuario_id` PK FK	uuid	não		Dono das preferências (também a chave primária) → usuarios.id
`tema`	text	não	'sistema'	claro, escuro ou sistema
`paleta`	text	não	'bento'	Uma das 16 paletas da interface

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`densidade`	text	não	'confortavel'	confortavel ou compacta
`sidebar_recolhida`	boolean	não	false	Menu lateral recolhido
`atualizado_em`	timestamptz	não	now()	Última alteração
`pagina_inicial` 1.4.0	text	não	'visao-geral'	Página aberta depois do login: visao-geral, analytics, usuarios, transacoes, notificacoes, relatorios ou auditoria
`itens_por_pagina` 1.4.0	smallint	não	25	Linhas por página nas tabelas: 10, 25, 50 ou 100
`movimento` 1.4.0	text	não	'sistema'	sistema segue o sistema operacional; reduzido desliga animações
`atalhos_teclado` 1.4.0	boolean	não	true	Liga os atalhos de uma tecla (/, ? e g + letra)
`primeiros_passos_ocultos` 1.4.0	boolean	não	false	Esconde o guia de primeiros passos da visão geral

Restrições

Tipo	Regra
verificação	`densidade in ('confortavel', 'compacta')`
verificação	`itens_por_pagina in (10, 25, 50, 100)`
verificação	`movimento in ('sistema', 'reduzido')`
verificação	`pagina_inicial in ('visao-geral', 'analytics', 'usuarios', 'transacoes', 'notificacoes', 'relatorios', 'auditoria')`
verificação	`paleta in ('bento', 'oliva', 'grafite', 'oceano', 'terracota', 'ameixa', 'floresta', 'ardosia', 'vinho', 'lavanda', 'cobre', 'petroleo', 'girassol', 'menta', 'noite', 'areia')`
verificação	`tema in ('claro', 'escuro', 'sistema')`
chave estrangeira	`usuario_id` → `usuarios.id` , ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`preferencias_usuario_pkey`	`(usuario_id)`	único	Chave primária

3.7 preferencias_notificacao

Categorias de notificação que cada pessoa recebe. Sem linha para a categoria, a pessoa recebe no painel.

4 campos. 1 índice e 0 linhas na demonstração

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`usuario_id` PK FK	uuid	não		Dono da preferência → usuarios.id
`categoria` PK	text	não		transacao, usuario, sistema, seguranca ou relatorio
`app`	boolean	não	true	Recebe no painel e em tempo real
`email`	boolean	não	false	Preferência de e-mail, guardada para quem integrar um serviço de envio

Restrições

Tipo	Regra
verificação	`categoria in ('transacao', 'usuario', 'sistema', 'seguranca', 'relatorio')`
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`preferencias_notificacao_pkey`	`(usuario_id, categoria)`	único	Chave primária

3.8 sessoes

requisição confere a sessão no banco: bloquear uma conta, trocar a senha ou encerrar uma sessão vale na hora, sem esperar o JWT expirar. 9.3 Segredos só como hash ou cifrados Senhas usam bcrypt. Tokens de renovação e chaves de API usam SHA-256, suficiente para valores aleatórios longos e rápido de conferir. Segredos de integrações precisam ser lidos de volta para os envios, então são cifrados

10 campos. 4 índices e 0 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador da sessão, levado no JWT de acesso
`usuario_id` FK	uuid	não		Dono da sessão → usuarios.id
`token_hash`	text	não		
`agente`	text	sim		User-Agent do navegador
`ip`	text	sim		IP do login
`lembrar`	boolean	não	false	Sessão longa (30 dias) ou curta (12 horas)
`criado_em`	timestamptz	não	now()	Momento do login
`ultimo_uso`	timestamptz	não	now()	Última renovação

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`expira_em`	timestamptz	não		Fim da validade
`revogada_em`	timestamptz	sim		Preenchido ao sair, ao encerrar a sessão, ao trocar a senha ou ao bloquear a conta

Restrições

Tipo	Regra
verificação	`expira_em > criado_em`
único	`(token_hash)`
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`sessoes_pkey`	`(id)`	único	Chave primária
`sessoes_expira_em`	`(expira_em)`	comum	Limpeza das sessões expiradas pela retenção
`sessoes_token_hash_key`	`(token_hash)`	único	Unicidade
`sessoes_usuario_ativas`	`(usuario_id) onde revogada_em IS NULL`	parcial	Sessões ativas de uma pessoa

3.9 chaves_api

Chaves para integrar outros sistemas. O segredo aparece uma vez ao criar e é guardado só como hash.

9 campos. 3 índices e 0 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`usuario_id` FK	uuid	não		Dono; a chave age com as permissões dele → usuarios.id
`nome`	text	não		Nome dado à chave (2 a 60 caracteres)
`prefixo`	text	não		Começo visível da chave, para reconhecê-la na lista
`segredo_hash`	text	não		SHA-256 do segredo bnt_...
`escopos`	text[]	não		leitura e/ou escrita; leitura libera só GET
`ultimo_uso`	timestamptz	sim		Último pedido feito com a chave

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`criado_em`	timestamptz	não	now()	Data de criação
`revogada_em`	timestamptz	sim		Preenchido ao revogar

Restrições

Tipo	Regra
verificação	`(cardinality(escopos) > 0) AND (escopos <@ ARRAY['leitura', 'escrita'])`
verificação	`(char_length(btrim(nome)) >= 2) AND (char_length(btrim(nome)) <= 60)`
único	`(segredo_hash)`
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`chaves_api_pkey`	`(id)`	único	Chave primária
`chaves_api_segredo_hash_key`	`(segredo_hash)`	único	Unicidade
`chaves_api_usuario`	`(usuario_id) onde revogada_em IS NULL`	parcial	Chaves ativas de uma pessoa

3.10 notas_usuario

Notas internas que a equipe escreve no perfil de uma conta. Ficam fora da tabela usuarios porque cada conta pode ter várias, com autor e data próprios. Criada pela migração 005.

7 campos. 2 índices e 0 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador da nota
`usuario_id` FK	uuid	não		Conta a que a nota pertence → usuarios.id
`autor_id` FK	uuid	sim		Quem escreveu; nulo se a conta do autor foi excluída → usuarios.id
`texto`	text	não		Conteúdo da nota, de 2 a 2.000 caracteres
`fixada`	boolean	não	false	Nota destacada no topo
`criado_em`	timestamptz	não	now()	Data de criação
`atualizado_em`	timestamptz	não	now()	Última edição do texto (fixar não altera)

Copyright (©) 2026 Erik Pimenta

Restrições

Tipo	Regra
verificação	`(char_length(btrim(texto)) >= 2) AND (char_length(btrim(texto)) <= 2000)`
chave estrangeira	`autor_id` → `usuarios.id`, ao excluir: vira nulo, ao atualizar: impede
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`notas_usuario_pkey`	`(id)`	único	Chave primária
`notas_usuario_lista`	`(usuario_id, fixada DESC, criado_em DESC)`	comum	Lista da conta na ordem da tela, fixadas primeiro, sem ordenação extra

Notas internas

Excluir um cliente apaga as notas dele em cascata; excluir alguém da equipe preserva o histórico que essa pessoa deixou, com `autor_id` nulo. A regra de autoria (só o autor ou quem tem `usuarios.permissoes` altera) fica na API, e a auditoria guarda só um trecho de até 80 caracteres da nota.

Copyright (©) 2026 Erik Pimenta

04

Negócio

Planos, assinaturas, transações e metas: a parte do banco que alimenta receita, MRR, conversão e cancelamentos.

4.1 planos

Planos vendidos pela plataforma. A semente cria Essencial, Profissional, Equipe e Empresa.

7 campos. 1 índice e 4 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`codigo` PK	text	não		Identificador do plano
`nome`	text	não		Nome exibido
`preco_mensal`	integer	não		Preço mensal em centavos
`preco_anual`	integer	não		Preço anual em centavos
`ordem`	smallint	não	0	Posição nas listas
`ativo`	boolean	não	true	Falso tira o plano de novas vendas
`recursos`	jsonb	não	'[]'	Lista de recursos em JSON

Restrições

Tipo	Regra
verificação	`codigo ~ '^[a-z][a-z0-9_]{1,31}$'`
verificação	`preco_anual > 0`
verificação	`preco_mensal > 0`

Índices

Índice	Colunas	Tipo	Para que serve
`planos_pkey`	`(codigo)`	único	Chave primária

Copyright (©) 2026 Erik Pimenta

4.2 assinaturas

Contratos dos clientes com os planos. Cada usuário tem no máximo uma assinatura vigente (ativa, em teste ou inadimplente), garantida por um índice único parcial.

15 campos. 7 índices e 1.780 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`usuario_id` FK	uuid	não		Cliente → usuarios.id
`plano` FK	text	não		Plano contratado → planos.codigo
`status`	text	não		ativa, teste, cancelada ou inadimplente
`ciclo`	text	não	'mensal'	mensal ou anual
`metodo`	text	não	'cartao'	cartao, pix ou boleto
`valor`	integer	não		Valor do ciclo em centavos
`iniciada_em`	timestamptz	não		Início do contrato (ou do teste)
`teste_ate`	timestamptz	sim		Fim do período de teste
`convertida_em`	timestamptz	sim		Quando passou de teste a pagante
`renovacao_em`	timestamptz	sim		Próxima cobrança
`cancelada_em`	timestamptz	sim		Data do cancelamento
`motivo_cancelamento`	text	sim		Motivo informado
`criado_em`	timestamptz	não	now()	Criação da linha
`atualizado_em`	timestamptz	não	now()	Última alteração

Restrições

Tipo	Regra
verificação	`(teste_ate IS NULL) OR (teste_ate >= iniciada_em)`
verificação	`(convertida_em IS NULL) OR (convertida_em >= iniciada_em)`
verificação	`(status = 'cancelada') = (cancelada_em IS NOT NULL)`
verificação	`(status <> 'teste') OR (teste_ate IS NOT NULL)`
verificação	`ciclo in ('mensal', 'anual')`
verificação	`metodo in ('cartao', 'pix', 'boleto')`
verificação	`status in ('ativa', 'teste', 'cancelada', 'inadimplente')`

Copyright (©) 2026 Erik Pimenta

Tipo	Regra
verificação	`valor >= 0`
chave estrangeira	`plano` → `planos.codigo`, ao excluir: impede, ao atualizar: cascata
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`assinaturas_pkey`	`(id)`	único	Chave primária
`assinaturas_cancelada_em`	`(cancelada_em) onde cancelada_em IS NOT NULL`	parcial	Cancelamentos por período (churn)
`assinaturas_convertida_em`	`(convertida_em) onde convertida_em IS NOT NULL`	parcial	Conversões por período
`assinaturas_iniciada_em`	`(iniciada_em)`	comum	Assinaturas novas por período
`assinaturas_plano`	`(plano)`	comum	Assinaturas por plano
`assinaturas_status`	`(status)`	comum	Contagem por status
`assinaturas_uma_vigente`	`(usuario_id) onde status in ('ativa', 'teste', 'inadimplente')`	único parcial	No máximo uma assinatura vigente por usuário

4.3 transacoes

Cobranças, upgrades, vendas avulsas e reembolsos. É a base da receita, do MRR, do ticket médio e dos relatórios financeiros. Desde a 1.1.0, guarda também quem reembolsou, quando e por quê.

17 campos. 8 índices e 9.431 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`codigo`	text	não	('TX-'|| lpad((nextval('transacoes_codigo_seq')), 6, '0'))	
`usuario_id` FK	uuid	sim		Cliente; vira nulo se a conta for excluída → usuarios.id
`assinatura_id` FK	uuid	sim		Assinatura cobrada, quando houver → assinaturas.id
`plano` FK	text	sim		Plano da cobrança → planos.codigo
`cliente_nome`	text	não		Cópia do nome no momento da cobrança

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`cliente_email`	text	não		Cópia do e-mail no momento da cobrança
`tipo`	text	não		assinatura, upgrade, avulsa ou reembolso
`metodo`	text	não		cartao, pix ou boleto
`status`	text	não		paga, pendente, falhou ou reembolsada
`valor`	integer	não		Valor em centavos
`moeda`	character(3)	não	'BRL'	Código ISO 4217, padrão BRL
`criado_em`	timestamptz	não	now()	Criação da cobrança
`pago_em`	timestamptz	sim		Preenchido quando paga ou reembolsada
`reembolsado_em`	timestamptz	sim		Novo na 1.1.0. Momento do reembolso feito pelo painel ou pela API
`reembolso_motivo`	text	sim		Novo na 1.1.0. Motivo informado ao reembolsar (3 a 300 caracteres)
`reembolsado_por` FK	uuid	sim		Novo na 1.1.0. Quem reembolsou; vira nulo se a → usuarios.id

Restrições

Tipo	Regra
verificação	`(status in ('paga', 'reembolsada')) = (pago_em IS NOT NULL)`
verificação	`metodo in ('cartao', 'pix', 'boleto')`
verificação	`(reembolso_motivo IS NULL) OR (char_length(reembolso_motivo) >= 3) AND (char_length(reembolso_motivo) <= 300)`
verificação	`status in ('paga', 'pendente', 'falhou', 'reembolsada')`
verificação	`tipo in ('assinatura', 'upgrade', 'avulsa', 'reembolso')`
verificação	`valor >= 0`
único	`(codigo)`
chave estrangeira	`assinatura_id` → `assinaturas.id`, ao excluir: vira nulo, ao atualizar: impede
chave estrangeira	`plano` → `planos.codigo`, ao excluir: impede, ao atualizar: cascata
chave estrangeira	`reembolsado_por` → `usuarios.id`, ao excluir: vira nulo, ao atualizar: impede

Copyright (©) 2026 Erik Pimenta

Tipo	Regra
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: vira nulo, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`transacoes_pkey`	`(id)`	único	Chave primária
`transacoes_assinatura`	`(assinatura_id)`	comum	Cobranças de uma assinatura
`transacoes_codigo_key`	`(codigo)`	único	Unicidade
`transacoes_criado_em`	`(criado_em DESC)`	comum	Listagem das mais recentes
`transacoes_metodo`	`(metodo)`	comum	Filtro por método
`transacoes_plano`	`(plano, criado_em)`	comum	Receita por plano e período
`transacoes_status_criado_em`	`(status, criado_em)`	comum	Receita paga por período e filtro por status na página Transações
`transacoes_usuario`	`(usuario_id, criado_em DESC)`	comum	Histórico de um cliente

4.4 metas_mensais

Meta de receita de cada mês, usada no medidor da visão geral e na linha de meta do gráfico mensal.

2 campos. 1 índice e 25 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`mes` PK	date	não		Primeiro dia do mês
`valor`	integer	não		Meta em centavos

Restrições

Tipo	Regra
verificação	`EXTRACT(day FROM mes) = 1`
verificação	`valor > 0`

Índices

Índice	Colunas	Tipo	Para que serve
`metas_mensais_pkey`	`(mes)`	único	Chave primária

Copyright (©) 2026 Erik Pimenta

05

Tráfego

Visitantes, cadastros e conversões por dia, canal, dispositivo e país, usados no funil e nos gráficos de analytics.

5.1 trafego_diario

Tráfego agregado por dia, canal e dispositivo. Alimenta visitantes, conversão, funil, canais e dispositivos do analytics.

7 campos. 2 índices e 13.137 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`dia` PK	date	não		Dia de calendário
`canal` PK	text	não		Orgânico, Direto, Pago, Social, Indicação ou E-mail
`dispositivo` PK	text	não		Desktop, Celular ou Tablet
`visitantes`	integer	não	0	Visitantes únicos
`sessoes`	integer	não	0	Sessões de navegação
`cadastros`	integer	não	0	Contas criadas
`conversoes`	integer	não	0	Assinaturas iniciadas

Restrições

Tipo	Regra
verificação	`cadastros >= 0`
verificação	`canal in ('Orgânico', 'Direto', 'Pago', 'Social', 'Indicação', 'E-mail')`
verificação	`conversoes >= 0`
verificação	`dispositivo in ('Desktop', 'Celular', 'Tablet')`
verificação	`sessoes >= 0`
verificação	`visitantes >= 0`

Copyright (©) 2026 Erik Pimenta

Índices

Índice	Colunas	Tipo	Para que serve
`trafego_diario_pkey`	`(dia, canal, dispositivo)`	único	Chave primária
`trafego_diario_canal`	`(canal, dia)`	comum	Séries por canal

5.2 trafego_paises

Visitantes por dia e país, para o ranking de países do analytics.

3 campos. 1 índice e 7.310 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`dia` PK	date	não		Dia de calendário
`pais_codigo` PK	character(2)	não		País em ISO 3166-1 alfa-2
`visitantes`	integer	não	0	Visitantes únicos

Restrições

Tipo	Regra
verificação	`pais_codigo ~ '^[A-Z]{2}$'`
verificação	`visitantes >= 0`

Índices

Índice	Colunas	Tipo	Para que serve
`trafego_paises_pkey`	`(dia, pais_codigo)`	único	Chave primária

Copyright (©) 2026 Erik Pimenta

06

Comunicação e relatórios

Notificações, leituras, relatórios gerados e envios agendados.

6.1 notificacoes

em , criadas só quando a pessoa interage. Assim, um aviso para cem pessoas não gera cem notificacao_estados linhas.

10 campos. 3 índices e 60 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`destinatario_id` FK	uuid	sim		Pessoa que recebe; nulo envia para toda a equipe → usuarios.id
`tipo`	text	não		info, sucesso, alerta ou erro
`categoria`	text	não		transacao, usuario, sistema, seguranca ou relatorio
`titulo`	text	não		Título (até 120 caracteres)
`mensagem`	text	não		Texto (até 1.000 caracteres)
`link`	text	sim		Endereço aberto ao clicar
`dados`	jsonb	não	'{}'	Dados extras em JSON, como ids relacionados
`criado_por` FK	uuid	sim		Autor de um aviso manual → usuarios.id
`criado_em`	timestamptz	não	now()	Momento da criação

Restrições

Tipo	Regra
verificação	`categoria in ('transacao', 'usuario', 'sistema', 'seguranca', 'relatorio')`
verificação	`(char_length(mensagem) >= 1) AND (char_length(mensagem) <= 1000)`
verificação	`tipo in ('info', 'sucesso', 'alerta', 'erro')`
verificação	`(char_length(titulo) >= 1) AND (char_length(titulo) <= 120)`
chave estrangeira	`criado_por` → `usuarios.id`, ao excluir: vira nulo, ao atualizar: impede

Copyright (©) 2026 Erik Pimenta

Tipo	Regra
chave estrangeira	`destinatario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`notificacoes_pkey`	`(id)`	único	Chave primária
`notificacoes_criado_em`	`(criado_em DESC)`	comum	Avisos gerais em ordem de chegada
`notificacoes_destinatario`	`(destinatario_id, criado_em DESC)`	comum	Caixa de cada pessoa em ordem de chegada

6.2 notificacao_estados

sem antes exportá-la (página Auditoria ou). GET /api/v1/atividades/exportar 12.3 Evoluindo o esquema 1 Crie com as instruções. Para mudar um domínio fechado, apague e api/migracoes/005_descricao.sql recrie a regra de verificação: .

4 campos. 2 índices e 188 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`notificacao_id` PK FK	uuid	não		Notificação → notificacoes.id
`usuario_id` PK FK	uuid	não		Pessoa → usuarios.id
`lida_em`	timestamptz	sim		Quando a pessoa leu
`dispensada_em`	timestamptz	sim		Quando a pessoa dispensou

Restrições

Tipo	Regra
chave estrangeira	`notificacao_id` → `notificacoes.id`, ao excluir: cascata, ao atualizar: impede
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`notificacao_estados_pkey`	`(notificacao_id, usuario_id)`	único	Chave primária
`notificacao_estados_usuario`	`(usuario_id)`	comum	Contagem de não lidas

Copyright (©) 2026 Erik Pimenta

6.3 relatorios_agendados

Envios recorrentes de relatórios. O agendador interno gera os vencidos a cada minuto e calcula a próxima execução.

11 campos. 2 índices e 2 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`nome`	text	não		Nome do agendamento
`tipo`	text	não		receita, transacoes, usuarios, assinaturas ou crescimento
`formato`	text	não		csv ou pdf
`frequencia`	text	não		diaria, semanal (segunda-feira) ou mensal (dia 1º)
`periodo`	text	não		7d, 30d ou mes-anterior
`ativo`	boolean	não	true	Falso pausa o agendamento
`proxima_execucao`	timestamptz	não		Próxima execução, sempre às 7h no fuso configurado
`ultima_execucao`	timestamptz	sim		Última execução
`criado_por` FK	uuid	não		Quem criou e recebe o aviso → usuarios.id
`criado_em`	timestamptz	não	now()	Data de criação

Restrições

Tipo	Regra
verificação	`formato in ('csv', 'pdf')`
verificação	`frequencia in ('diaria', 'semanal', 'mensal')`
verificação	`(char_length(btrim(nome)) >= 2) AND (char_length(btrim(nome)) <= 120)`
verificação	`periodo in ('7d', '30d', 'mes-anterior')`
verificação	`tipo in ('receita', 'transacoes', 'usuarios', 'assinaturas', 'crescimento')`
chave estrangeira	`criado_por` → `usuarios.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`relatorios_agendados_pkey`	`(id)`	único	Chave primária

Copyright (©) 2026 Erik Pimenta

Índice	Colunas	Tipo	Para que serve
`relatorios_agendados_vencidos`	`(proxima_execucao) onde ativo`	parcial	Agendador encontra os envios vencidos

6.4 relatorios

e atualiza . agendamento_id ultima_execucao 4 Cria uma notificação para e dispara para as integrações. criado_por relatorio.gerado

16 campos. 2 índices e 3 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`nome`	text	não		Nome do relatório
`tipo`	text	não		receita, transacoes, usuarios, assinaturas ou crescimento
`formato`	text	não		csv ou pdf
`inicio`	date	não		Primeiro dia do período
`fim`	date	não		Último dia do período
`filtros`	jsonb	não	'{}'	Filtros usados, em JSON
`status`	text	não		pronto ou erro
`linhas`	integer	não	0	Linhas de dados no arquivo
`tamanho_bytes`	integer	não	0	Tamanho do arquivo
`conteudo`	bytea	sim		Arquivo gerado
`tipo_mime`	text	não		text/csv ou application/pdf
`erro`	text	sim		Mensagem quando a geração falha
`criado_por` FK	uuid	sim		Quem gerou; nulo se a conta for excluída → usuarios.id
`agendamento_id` FK	uuid	sim		Agendamento que gerou, quando houver →
`criado_em`	timestamptz	não	now()	Momento da geração

Restrições

Tipo	Regra
verificação	`fim >= inicio`
verificação	`(status = 'pronto') = (conteudo IS NOT NULL)`

Copyright (©) 2026 Erik Pimenta

Tipo	Regra
verificação	`formato in ('csv', 'pdf')`
verificação	`linhas >= 0`
verificação	`(char_length(btrim(nome)) >= 2) AND (char_length(btrim(nome)) <= 160)`
verificação	`status in ('pronto', 'erro')`
verificação	`tamanho_bytes >= 0`
verificação	`tipo in ('receita', 'transacoes', 'usuarios', 'assinaturas', 'crescimento')`
chave estrangeira	`agendamento_id` → `relatorios_agendados.id`, ao excluir: vira nulo, ao atualizar: impede
chave estrangeira	`criado_por` → `usuarios.id`, ao excluir: vira nulo, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`relatorios_pkey`	`(id)`	único	Chave primária
`relatorios_criado_em`	`(criado_em DESC)`	comum	Histórico de relatórios

Copyright (©) 2026 Erik Pimenta

07

Sistema

Integrações e suas entregas, auditoria, configurações gerais e o controle das migrações.

7.1 integracoes

Uma linha por provedor. Os campos de configuração ficam em JSON, com os segredos cifrados.

12 campos. 2 índices e 5 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador
`provedor`	text	não		
`nome`	text	não		Nome exibido
`descricao`	text	não	''	Descrição exibida no cartão
`categoria`	text	não		Automação, Comunicação, Analytics ou Pagamentos
`ativo`	boolean	não	false	Recebe eventos quando verdadeiro
`config`	jsonb	não	'{}'	Campos da integração; segredos cifrados com AES- 256-GCM (prefixo enc:v1:)
`status`	text	não	'desconectada'	conectada, desconectada ou erro
`ultimo_evento`	timestamptz	sim		Último envio bem-sucedido
`ultimo_erro`	text	sim		Última mensagem de erro
`criado_em`	timestamptz	não	now()	Criação
`atualizado_em`	timestamptz	não	now()	Última alteração

Restrições

Tipo	Regra
verificação	`categoria in ('Automação', 'Comunicação', 'Analytics', 'Pagamentos')`
verificação	`provedor in ('webhook', 'slack', 'discord', 'google-analytics', 'stripe')`

Copyright (©) 2026 Erik Pimenta

Tipo	Regra
verificação	`status in ('conectada', 'desconectada', 'erro')`
único	`(provedor)`

Índices

Índice	Colunas	Tipo	Para que serve
`integracoes_pkey`	`(id)`	único	Chave primária
`integracoes_provedor_key`	`(provedor)`	único	Unicidade

7.2 entregas_integracao

Histórico e fila das entregas de eventos para as integrações. Cada evento enviado vira uma linha; as pendentes são a fila de novas tentativas que o agendador processa a cada minuto. Criada pela migração 006. (Nova na 1.4.0.)

14 campos. 3 índices e 0 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	uuid	não	gen_random_uuid()	Identificador da entrega
`integracao_id` FK	uuid	não		Integração de destino → integracoes.id
`evento`	text	não		Nome do evento, como transacao.paga
`resumo`	text	não		Frase curta mostrada no histórico
`carga`	jsonb	não	'{}'	Dados do evento enviados no corpo, sem segredos
`status`	text	não	'pendente'	pendente, entregue ou falhou
`tentativas`	smallint	não	0	Quantas vezes o envio foi tentado
`proxima_tentativa`	timestamptz	sim		Quando sai a próxima tentativa; só existe enquanto pendente
`codigo_http`	smallint	sim		Código HTTP da última resposta do destino
`duracao_ms`	integer	sim		Tempo da última tentativa, em milissegundos
`erro`	text	sim		Mensagem da última falha, até 300 caracteres
`teste`	boolean	não	false	Entrega criada pelo botão Testar; não é repetida
`criado_em`	timestamptz	não	now()	Quando o evento entrou na fila
`atualizado_em`	timestamptz	não	now()	Última mudança de status

Copyright (©) 2026 Erik Pimenta

Restrições

Tipo	Regra
verificação	`(status = 'pendente') = (proxima_tentativa IS NOT NULL)`
verificação	`(duracao_ms IS NULL) OR (duracao_ms >= 0)`
verificação	`(erro IS NULL) OR (char_length(erro) <= 300)`
verificação	`evento ~ '^[a-z_]+\.[a-z_]+$'`
verificação	`(char_length(resumo) >= 1) AND (char_length(resumo) <= 300)`
verificação	`status in ('pendente', 'entregue', 'falhou')`
verificação	`tentativas >= 0`
chave estrangeira	`integracao_id` → `integracoes.id`, ao excluir: cascata, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`entregas_integracao_pkey`	`(id)`	único	Chave primária
`entregas_integracao_lista`	`(integracao_id, criado_em DESC)`	comum	Histórico de uma integração, mais recentes primeiro
`entregas_integracao_pendentes`	`(proxima_tentativa) onde status = 'pendente'`	parcial	Fila de novas tentativas: só as pendentes, pela hora da próxima tentativa

7.3 auditoria

. usuario.bloqueado 10.5 Relatório agendado 1 A cada minuto, o agendador busca em os ativos com vencida. relatorios_agendados proxima_execucao 2 Para cada um, reserva a execução atualizando só se ela ainda tiver o valor lido.

9 campos. 5 índices e 331 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`id` PK	bigint	não	identidade	Sequencial gerado pelo banco
`usuario_id` FK	uuid	sim		Autor; nulo para eventos do sistema → usuarios.id
`acao`	text	não		entidade.verbo, como usuario.bloqueado ou transacao.reembolsar
`entidade`	text	sim		Tipo do registro afetado
`entidade_id`	text	sim		Identificador do registro afetado

Copyright (©) 2026 Erik Pimenta

Campo	Tipo	Nulo	Padrão	Descrição
`descricao`	text	não		Frase pronta em português exibida nas atividades
`detalhes`	jsonb	não	'{}'	Dados extras em JSON, como o motivo de um reembolso
`ip`	text	sim		IP de origem
`criado_em`	timestamptz	não	now()	Momento do evento

Restrições

Tipo	Regra
verificação	`acao ~ '^[a-z_]+\.[a-z_]+$'`
chave estrangeira	`usuario_id` → `usuarios.id`, ao excluir: vira nulo, ao atualizar: impede

Índices

Índice	Colunas	Tipo	Para que serve
`auditoria_pkey`	`(id)`	único	Chave primária
`auditoria_acao_criado_em`	`(acao, criado_em DESC)`	comum	Novo na 1.1.0. Filtro por tipo de ação na página Auditoria, mais recentes primeiro
`auditoria_criado_em`	`(criado_em DESC)`	comum	Atividades recentes
`auditoria_entidade`	`(entidade, entidade_id)`	comum	Histórico de um registro
`auditoria_usuario`	`(usuario_id, criado_em DESC)`	comum	Atividades de uma pessoa

7.4 configuracoes_sistema

Configurações gerais em pares chave e valor: nomeEmpresa, moeda, fuso, metaMensal e idiomaPadrao. Na 1.4.0 passou a guardar também o estado do modo de manutenção (chave manutencao) e as reservas das tarefas automáticas (tarefa:limpeza e tarefa:backup).

3 campos. 1 índice e 6 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`chave` PK	text	não		Nome da configuração
`valor`	jsonb	não		Valor em JSON
`atualizado_em`	timestamptz	não	now()	Última alteração

Copyright (©) 2026 Erik Pimenta

Índices

Índice	Colunas	Tipo	Para que serve
`configuracoes_sistema_pkey`	`(chave)`	único	Chave primária

7.5 migracoes

Controle das migrações aplicadas. Criada pela própria API antes da primeira migração.

2 campos. 1 índice e 6 linhas na demonstração

Campo	Tipo	Nulo	Padrão	Descrição
`nome` PK	text	não		Arquivo de migração aplicado
`aplicada_em`	timestamptz	não	now()	Momento da aplicação

Índices

Índice	Colunas	Tipo	Para que serve
`migracoes_pkey`	`(nome)`	único	Chave primária

Chaves de configuracoes_sistema

`nomeEmpresa`, `moeda`, `fuso`, `metaMensal` e `idiomaPadrao` vêm da migração 003. `manutencao` guarda `{ ativo, mensagem, desde, porId, porNome }`. `tarefa:limpeza` e `tarefa:backup` guardam quando cada tarefa automática rodou pela última vez; o agendador só roda a tarefa se conseguir atualizar essa linha, o que garante uma execução por intervalo mesmo com várias instâncias.

Copyright (©) 2026 Erik Pimenta

08

Consultas e índices

As métricas do painel são calculadas na hora, direto das tabelas. Esta é a relação entre as consultas mais frequentes e os índices que as atendem.

8.1 Consultas principais

Onde aparece	Consulta	Índice usado
Receita, MRR e ticket médio	Transações pagas em um intervalo de criação	`transacoes_status_criado_em`
Página Transações	Filtro por status e período, mais recentes primeiro; busca por código, cliente ou e-mail	`transacoes_status_criado_em` `transacoes_criado_em` `transacoes_codigo_key`
Receita por plano	Transações por plano e período	`transacoes_plano`
Assinaturas novas, conversões e churn	Assinaturas por data de início, conversão e cancelamento	`assinaturas_iniciada_em` `assinaturas_convertida_em` `assinaturas_cancelada_em`
Usuários novos e ativos	Cadastros e últimos acessos por período	`usuarios_criado_em` `usuarios_ultimo_acesso`
Lista de usuários	Filtros por status e papel, ordenação por nome e datas	`usuarios_status` `usuarios_papel` `usuarios_nome`
Login e importação	Conta pelo e-mail sem diferenciar maiúsculas; na importação, `on conflict` no mesmo índice ignora os e-mails já cadastrados	`usuarios_email_unico`
Renovação de sessão	Sessão pelo hash do token	`sessoes_token_hash_key`
Chave de API	Chave pelo hash do segredo	`chaves_api_segredo_hash_key`
Central de notificações	Notificações da pessoa e de difusão, mais recentes primeiro	`notificacoes_destinatario` `notificacoes_criado_em`
Auditoria	Últimos registros, por pessoa, por registro, por tipo de ação e período	`auditoria_criado_em` `auditoria_usuario` `auditoria_entidade` `auditoria_acao_criado_em`
Notas internas	Notas da conta, fixadas primeiro	`notas_usuario_lista`

Copyright (©) 2026 Erik Pimenta

Onde aparece	Consulta	Índice usado
Entregas das integrações	Histórico de uma integração; fila de novas tentativas vencidas	`entregas_integracao_lista` `entregas_integracao_pendentes`
Agendador	Agendamentos ativos com execução vencida	`relatorios_agendados_vencidos`
Retenção	Sessões expiradas, notificações, relatórios e entregas antigas	`sessoes_expira_em` `notificacoes_criado_em` `entregas_integracao_lista`
Analytics de tráfego	Tráfego por dia, canal e dispositivo	`trafego_diario_pkey` `trafego_diario_canal`
Status do sistema e métricas	Contagem de registros das principais tabelas e migrações aplicadas	`migracoes_pkey` (contagens leem a tabela inteira)

8.2 Índices parciais

Cinco grupos de índices cobrem só as linhas que interessam, o que os mantém pequenos mesmo com muito histórico:

- `assinaturas_uma_vigente`: único em `usuario_id` só para assinaturas ativas, em teste ou inadimplentes. É a regra de negócio “uma assinatura vigente por cliente” garantida pelo banco.
- `assinaturas_convertida_em` e `assinaturas_cancelada_em`: só as linhas em que a data existe.
- `sessoes_usuario_ativas` e `chaves_api_usuario`: só as não revogadas, usadas ao listar e ao encerrar sessões.
- `relatorios_agendados_vencidos`: só os agendamentos ativos, ordenados pela próxima execução.
- `entregas_integracao_pendentes`: só as entregas pendentes, pela hora da próxima tentativa. A fila continua pequena mesmo com milhões de entregas no histórico.

8.3 Desempenho medido

Com a semente de demonstração (cerca de 2.400 usuários, 1.800 assinaturas e 9.400 transações em 24 meses), a visão geral, o analytics e as listas de transações e auditoria respondem em dezenas de milissegundos no PGlite. A semente inteira é criada em poucos segundos, com inserções em lote, e um backup completo dela tem menos de 2 MB compactado. Para conferir o plano de uma consulta no seu banco, rode-a com `explain analyze` no `psql`. O capítulo 11 mostra o que fazer quando o volume crescer.

Copyright (©) 2026 Erik Pimenta

09

Decisões de arquitetura

As escolhas abaixo explicam por que o banco é como é. Cada uma resolve um problema concreto do painel.

9.1 Um código, dois motores

Rodar com PGlite quando não há `DATABASE_URL` permite baixar o projeto e usar em segundos, sem instalar banco. Como o PGlite é o próprio PostgreSQL, não existe uma segunda versão do SQL para manter, e os testes automatizados usam um banco real em memória, isolado por arquivo de teste.

9.2 Sessões revogáveis

O login gera um JWT de acesso de 15 minutos e um token de renovação aleatório de 48 bytes. Só o hash SHA-256 do token vai para `sessoes`, e cada renovação troca o token, então um token antigo roubado não funciona. Cada requisição confere a sessão no banco: bloquear uma conta, trocar a senha ou encerrar uma sessão vale na hora, sem esperar o JWT expirar.

9.3 Segredos só como hash ou cifrados

Senhas usam bcrypt. Tokens de renovação e chaves de API usam SHA-256, suficiente para valores aleatórios longos e rápido de conferir. Segredos de integrações precisam ser lidos de volta para os envios, então são cifrados com AES-256-GCM usando uma chave derivada de `BENTO_SEGREDO_JWT`; a API devolve só os quatro últimos caracteres. A carga guardada em `entregas_integracao` é só o evento, nunca a configuração da integração.

9.4 Tempo real pelo próprio banco

Eventos de negócio são publicados com `pg_notify` no canal `bento_eventos`. Cada instância da API mantém uma conexão com `LISTEN` e repassa os eventos aos navegadores conectados nela por Server-Sent Events. Não é preciso Redis nem fila externa para rodar várias instâncias. O mesmo canal leva o evento `sistema`, que liga e desliga o modo de manutenção em todas as instâncias ao mesmo tempo. Cargas acima de cerca de 7,5 kB, perto do limite do `NOTIFY`, são entregues só na instância local.

9.5 Difusão barata de avisos

Um aviso para toda a equipe é uma única linha em `notificacoes`, com destinatário nulo. Leitura e dispensa ficam em `notificacao_estados`, criadas só quando a pessoa interage. Assim, um aviso para cem pessoas não gera cem linhas.

Copyright (©) 2026 Erik Pimenta

9.6 Relatórios dentro do banco

O arquivo de cada relatório fica na coluna `conteudo`, do tipo `bytea`. Qualquer instância entrega o download sem disco compartilhado, e o backup do banco leva os relatórios junto. Os arquivos típicos têm de 3 a 110 kB, e a retenção apaga os mais antigos que `BENTO_RETENCAO_RELATORIOS`. O agendador reserva cada execução com `update ... where id = $1 and proxima_execucao = $2`: só a instância que conseguir a atualização gera o arquivo.

9.7 Histórico que sobrevive

Excluir um usuário não apaga transações, reembolsos, relatórios, avisos nem auditoria: as chaves viram nulas, e as transações mantêm a cópia do nome e do e-mail do cliente. A auditoria guarda a frase pronta em português, então as atividades continuam legíveis mesmo sem o autor. As tarefas automáticas registram as próprias ações com `usuario_id` nulo.

9.8 Reembolso na própria transação

O reembolso feito pelo painel muda a linha original para `reembolsada` e grava quem, quando e por quê, em vez de criar uma segunda transação. Assim o vínculo com a cobrança original nunca se perde, os relatórios de receita continuam somando reembolsos pelo status, e a regra `status in ('paga', 'reembolsada') = (pago_em is not null)` segue válida.

9.9 Regras no banco, lógica na aplicação

O banco garante o que nunca pode ser violado: tipos, chaves, valores permitidos, coerência entre status e datas, o tamanho dos textos livres e a assinatura vigente única. O resto, como cálculo de métricas, permissões, notificações e a política de novas tentativas, fica no TypeScript, onde é testado.

9.10 Fila de entregas no próprio banco

As entregas das integrações usam `entregas_integracao` como fila: a linha nasce `pendente` com `proxima_tentativa = now()`, o agendador pega as vencidas pelo índice parcial e grava o resultado. Como a reserva é feita no banco, várias instâncias dividem a fila sem enviar o mesmo evento duas vezes, e uma reinicialização no meio do caminho não perde nada: a entrega continua pendente e sai no próximo ciclo. A regra `(status = 'pendente') = (proxima_tentativa is not null)` impede uma entrega pendente sem horário ou uma concluída com tentativa marcada.

9.11 Backup lógico portável

O backup do Bento é lógico: cada linha vira JSON com `to_jsonb`, gravado em lotes de NDJSON compactado, dentro de uma transação somente leitura em `repeatable read`. A restauração recria o esquema com as migrações que o backup tinha, insere as linhas com `jsonb_populate_recordset` na ordem das chaves estrangeiras, acerta as sequências com `setval` e só então aplica as migrações mais novas. Por isso o mesmo arquivo vai do PGlite para o PostgreSQL e de uma versão do Bento para outra mais nova, sem depender da versão do `pg_dump`.

Copyright (©) 2026 Erik Pimenta

10

Fluxos de dados

Como as tabelas são usadas juntas nas operações mais importantes do painel.

10.1 Login e renovação

1. A API busca a conta em `usuarios` pelo e-mail em minúsculas, confere `status`, `bloqueio_temporario_ate` e o papel em `papeis.acesso_painel`.
2. Com senha errada, soma `tentativas_falhas`; na quinta, grava `bloqueio_temporario_ate` para daqui a 15 minutos, cria uma notificação de segurança e registra em `auditoria`.
3. Com senha certa, zera as tentativas, atualiza `ultimo_acesso`, cria a linha em `sessoes` com o hash do token e devolve os cookies. A resposta de `/auth/eu` lê também `preferencias_usuario`, que decide a página inicial.
4. Na renovação, a API acha a sessão por `token_hash`, confere validade e revogação, gera um token novo e troca o hash na mesma linha.

10.2 Transação paga

1. A transação entra em `transacoes` com `status = 'paga'` e `pago_em`, e a assinatura ligada recebe a nova `renovacao_em`.
2. Uma notificação de categoria `transacao` é criada em `notificacoes` com destinatário nulo.
3. A API publica os eventos `transacao`, `notificacao` e `contagem` em `bento_eventos`, e cada instância os entrega aos navegadores conectados.
4. Para cada integração ativa, uma linha entra em `entregas_integracao` com o evento `transacao.paga`, e o envio começa na hora (seção 10.6).

10.3 Reembolso

1. A rota confere a permissão `transacoes.reembolsar` e valida o motivo (3 a 300 caracteres).
2. Em uma transação do banco, `transacoes` recebe `status = 'reembolsada'`, `reembolsado_em`, `reembolso_motivo` e `reembolsado_por`, só se o status ainda for `'paga'`; se outra pessoa reembolsou antes, nada muda e a API responde 409.
3. A ação vai para `auditoria` como `transacao.reembolsar`, com o motivo em `detalhes`.
4. Uma notificação de categoria `transacao` é criada e o evento `transacao` é publicado em `bento_eventos`, atualizando as telas abertas.

Copyright (©) 2026 Erik Pimenta

10.4 Bloqueio de conta

1. Em uma transação, `usuarios` recebe `status = 'bloqueado'`, `bloqueado_em` e `motivo_bloqueio` (a regra de verificação exige os dois juntos).
2. Todas as linhas abertas de `sessoes` da pessoa recebem `revogada_em`, encerrando o acesso na próxima requisição.
3. A ação vai para `auditoria`, uma notificação de segurança é criada e as integrações recebem `usuario.bloqueado`.

10.5 Relatório agendado

1. A cada minuto, fora do modo de manutenção, o agendador busca em `relatorios_agendados` os ativos com `proxima_execucao` vencida.
2. Para cada um, reserva a execução atualizando `proxima_execucao` só se ela ainda tiver o valor lido.
3. Gera o arquivo do período (últimos 7 ou 30 dias, ou mês anterior), grava em `relatorios` com `agendamento_id` e atualiza `ultima_execucao`.
4. Cria uma notificação para `criado_por` e enfileira `relatorio.gerado` para as integrações.

10.6 Entrega de integração

1. O evento entra em `entregas_integracao` como `pendente`, com `tentativas = 0` e `proxima_tentativa = now()`.
2. O envio assina o corpo, espera até 5 segundos e grava `tentativas`, `codigo_http`, `duracao_ms` e, se falhar, `erro`.
3. Com resposta 2xx, a linha vira `entregue`, `proxima_tentativa` fica nula e `integracoes.ultimo_evento` é atualizado.
4. Com falha, a linha continua `pendente`, com `proxima_tentativa` daqui a 1, 5 ou 30 minutos. Na quarta falha, ou em um teste, vira `falhou` e `integracoes.ultimo_erro` recebe a mensagem.
5. O reenvio pelo painel volta a linha para `pendente` com `proxima_tentativa = now()` e registra `integracao.reenvio` em `auditoria`.

10.7 Modo de manutenção

1. `PUT /sistema/manutencao` grava o estado em `configuracoes_sistema` (chave `manutencao`) e, se mudou, registra `sistema.manutencao_ativada` ou `sistema.manutencao_encerrada` em `auditoria`, na mesma transação.
2. A API publica o evento `sistema` em `bento_eventos`; todas as instâncias atualizam o estado em memória e os navegadores mostram ou escondem a faixa.
3. Enquanto ativo, nenhuma alteração passa pela API, exceto as liberadas, e o agendador pula os relatórios agendados.

Copyright (©) 2026 Erik Pimenta

11

Escalabilidade

O esquema atende bem dezenas de milhares de clientes e milhões de transações com os índices atuais. Estes são os próximos passos quando o volume pedir.

11.1 Tabelas que mais crescem

Tabela	Crescimento	Quando agir	O que fazer
`transacoes`	Uma linha por cobrança	Alguns milhões de linhas	Particionar por mês com `partition by range (criado_em)` e arquivar partições antigas
`auditoria`	Uma linha por ação	Alguns milhões de linhas	Definir `BENTO_RETENCAO_AUDITORIA` (por exemplo, 730 dias) depois de exportar, ou particionar por mês
`entregas_integracao`	Uma linha por evento e integração	Com muitas integrações ativas	Reduzir `BENTO_RETENCAO_ENTREGAS`; a fila usa índice parcial e não cresce com o histórico
`trafego_diario`	Dias × canais × dispositivos	Depois de 24 meses	Consolidar em uma tabela mensal ou particionar por ano
`relatorios`	Arquivos em `bytea`	Histórico acima de alguns GB	Reduzir `BENTO_RETENCAO_RELATORIOS` ou mover o conteúdo para armazenamento de objetos e guardar só a chave
`sessoes` e `notificacoes`	Uma linha por login e por aviso	Sempre	Já resolvido pela retenção automática (seção 12.2)

11.2 Métricas pré-calculadas

Hoje as métricas são calculadas na hora. Com volume muito grande, crie uma visão materializada diária com receita, transações pagas, falhas, reembolsos e cadastros por dia, atualizada por um job, e aponte as consultas de período para ela:

Copyright (©) 2026 Erik Pimenta

```
create materialized view receita_diaria as
select (criado_em at time zone 'America/Sao_Paulo')::date as dia,
       count(*) filter (where status = 'paga') as pagas,
       coalesce(sum(valor) filter (where status = 'paga'), 0) as receita,
       coalesce(sum(valor) filter (where status = 'reembolsada'), 0) as reembolsos
  from transacoes
 group by 1;

create unique index on receita_diaria (dia);
refresh materialized view concurrently receita_diaria;
```

11.3 Busca textual

A busca de usuários, transações e auditoria usa `ilike`, rápida até centenas de milhares de linhas. Acima disso, no PostgreSQL, ative a extensão `pg_trgm` e crie índices GIN (a extensão não existe no PGlite):

```
create extension if not exists pg_trgm;
create index usuarios_busca on usuarios using gin (nome gin_trgm_ops, email gin_trgm_ops);
create index transacoes_busca on transacoes using gin (cliente_nome gin_trgm_ops, cliente_email
gin_trgm_ops);
create index auditoria_busca on auditoria using gin (descricao gin_trgm_ops);
```

11.4 Várias instâncias e réplicas

A API não guarda estado em memória além das conexões e do estado da manutenção, que vem do banco: sessões estão no banco, os eventos passam pelo `LISTEN` / `NOTIFY` e as tarefas automáticas se reservam em `configuracoes_sistema` e `relatorios_agendados`. Coloque quantas instâncias quiser atrás de um balanceador, com a verificação de saúde em `/api/v1/saude/pronto`.

Analytics, relatórios e exportações só leem dados; podem ir para uma réplica de leitura com uma segunda URL de conexão. Com PgBouncer em modo transação, deixe a conexão de `LISTEN` apontando direto para o PostgreSQL, porque ela precisa de uma sessão própria. Ajuste `BENTO_POOL` para que instâncias × conexões fique abaixo do `max_connections` do servidor.

Copyright (©) 2026 Erik Pimenta

12

Operação

Backups, restauração, retenção de dados e evolução do esquema no dia a dia.

12.1 Backup e restauração

O Bento tem um backup próprio, que funciona igual em PGlite e PostgreSQL:

```
npm run backup
npm run backup -- --pasta /mnt/copias
npm run restaurar -- dados/backups/bento-backup-20261104-031500.ndjson.gz
docker compose exec bento node api/dist/cli.js backup
```

O arquivo traz o cabeçalho (versão do Bento, migrações, contagem por tabela e sequências), os registros de cada tabela em lotes e uma linha de fim; um arquivo cortado é recusado sem alterar nada. Com `BENTO_BACKUP_AUTOMATICO=diario` ou `semanal`, o agendador grava um backup no intervalo e mantém os `BENTO_BACKUP_MANTER` mais recentes. A aba Configurações › Manutenção lista os arquivos e baixa cada um.

Com PostgreSQL, o `pg_dump` continua sendo uma boa segunda camada, por exemplo antes de uma atualização grande do servidor:

```
docker compose exec -T postgres pg_dump -U bento -d bento -Fc > bento-$(date +%F).dump
docker compose exec -T postgres pg_restore -U bento -d bento --clean --if-exists < bento-2026-11-04.dump
```

Guarde o segredo junto

Os segredos das integrações são cifrados com uma chave derivada de `BENTO_SEGREDO_JWT`. Um backup restaurado com outro segredo funciona, mas as integrações precisam ter os segredos informados de novo, e todas as sessões são encerradas.

12.2 Retenção automática

Uma vez por dia o agendador apaga o que passou do prazo. Antes da 1.4.0 isso exigia um job externo; agora as regras vêm das variáveis `BENTO_RETENCAO_*`:

Tabela	Prazo padrão	O que sai
`sessoes`	30 dias	Sessões revogadas ou expiradas há mais tempo que o prazo

Copyright (©) 2026 Erik Pimenta

Tabela	Prazo padrão	O que sai
`notificacoes`	180 dias	Notificações antigas; as linhas de `notificacao_estados` saem junto, em cascata
`relatorios`	90 dias	Relatórios gerados há mais tempo que o prazo, com o arquivo
`entregas_integracao`	30 dias	Entregas concluídas ou com falha; as pendentes nunca são apagadas
`auditoria`	sem limite	Só com `BENTO_RETENCAO_AUDITORIA` maior que zero

`npm run limpar -- --simular` mostra a contagem e o espaço de cada categoria sem apagar nada. Exporte a auditoria (página Auditoria ou `GET /api/v1/atividades/exportar`) antes de definir um prazo para ela.

12.3 Evoluindo o esquema

1. Crie `api/migracoes/007_descricao.sql` com as instruções. Para mudar um domínio fechado, apague e recrie a regra de verificação: `alter table ... drop constraint ..., add constraint ... check (...)`.
2. Rode `npm run migrar` em desenvolvimento e `npm test`: os testes aplicam todas as migrações em um banco novo, e o teste de backup confere que a tabela nova entra no backup e na restauração.
3. Atualize a validação Zod da rota e os tipos em TypeScript que usam a tabela.
4. Em produção, faça um backup, ligue o modo de manutenção se a mudança for longa e suba a nova versão: a migração roda sozinha, em uma transação, e a sonda de prontidão só responde 200 quando ela termina.

12.4 Consultas úteis

```
select relname as tabela, n_live_tup as linhas, pg_size_pretty(pg_total_relation_size(relid)) as
tamanho
  from pg_stat_user_tables order by pg_total_relation_size(relid) desc;

select nome, aplicada_em from migracoes order by nome;

select i.nome, e.status, count(*) as entregas, max(e.criado_em) as ultima
  from entregas_integracao e join integracoes i on i.id = e.integracao_id
 group by i.nome, e.status order by i.nome, e.status;

select chave, valor, atualizado_em from configuracoes_sistema
 where chave in ('manutencao', 'tarefa:limpeza', 'tarefa:backup');

select t.codigo, t.valor, t.reembolsado_em, u.nome as reembolsado_por, t.reembolso_motivo
  from transacoes t left join usuarios u on u.id = t.reembolsado_por
 where t.reembolsado_em is not null order by t.reembolsado_em desc limit 20;
```

Copyright (©) 2026 Erik Pimenta

13

Dados de demonstração

Em banco vazio, a API cria 24 meses de dados fictícios, para o painel já abrir com gráficos e tabelas cheios.

13.1 Volume

Contagem de uma instalação nova, logo depois da semente:

Tabela	Linhas	Tabela	Linhas
`trafego_diario`	13.137	integracoes	5
`transacoes`	9.431	papeis	5
`trafego_paises`	7.310	planos	4
`usuarios`	2.393	preferencias_usuario	4
`assinaturas`	1.780	relatorios	3
`auditoria`	331	relatorios_agendados	2
`notificacao_estados`	188	chaves_api	0
`notificacoes`	60	entregas_integracao	0
`papel_permissoes`	48	notas_usuario	0
`metas_mensais`	25	preferencias_notificacao	0
`permissoes`	19	sessoes	0
`configuracoes_sistema`	6	usuario_permissoes	0
`migracoes`	6		

Nomes e e-mails são fictícios, gerados a partir de listas de nomes e sobrenomes, com e-mails no domínio `exemplo.com`. As contagens de transações, tráfego e auditoria variam um pouco a cada semente, porque dependem da data de criação.

Copyright (©) 2026 Erik Pimenta

13.2 Contas da equipe

E-mail	Nome	Papel
admin@bento.local	Ana Martins	Administrador
gerente@bento.local	Bruno Costa	Gerente
analista@bento.local	Carla Dias	Analista
suporte@bento.local	Diego Rocha	Suporte

Todas usam a senha `bento123`. Enquanto alguma conta da equipe usar essa senha, o diagnóstico avisa (e, em produção, aponta erro) e o guia de primeiros passos mantém o item **Senhas de demonstração** aberto.

13.3 Desligando a demonstração

Com `BENTO_SEMEAR=nao`, o banco começa só com os dados base das migrações: papéis, permissões, planos, integrações desativadas e configurações. Defina `BENTO_ADMIN_EMAIL`, `BENTO_ADMIN_SENHA` e `BENTO_ADMIN_NOME` para criar o primeiro administrador e desligue o simulador com `BENTO_SIMULADOR=false`. Para descartar dados já criados, apague a pasta `dados/pglite` (PGlite) ou rode `docker compose down -v`.

Copyright (©) 2026 Erik Pimenta